

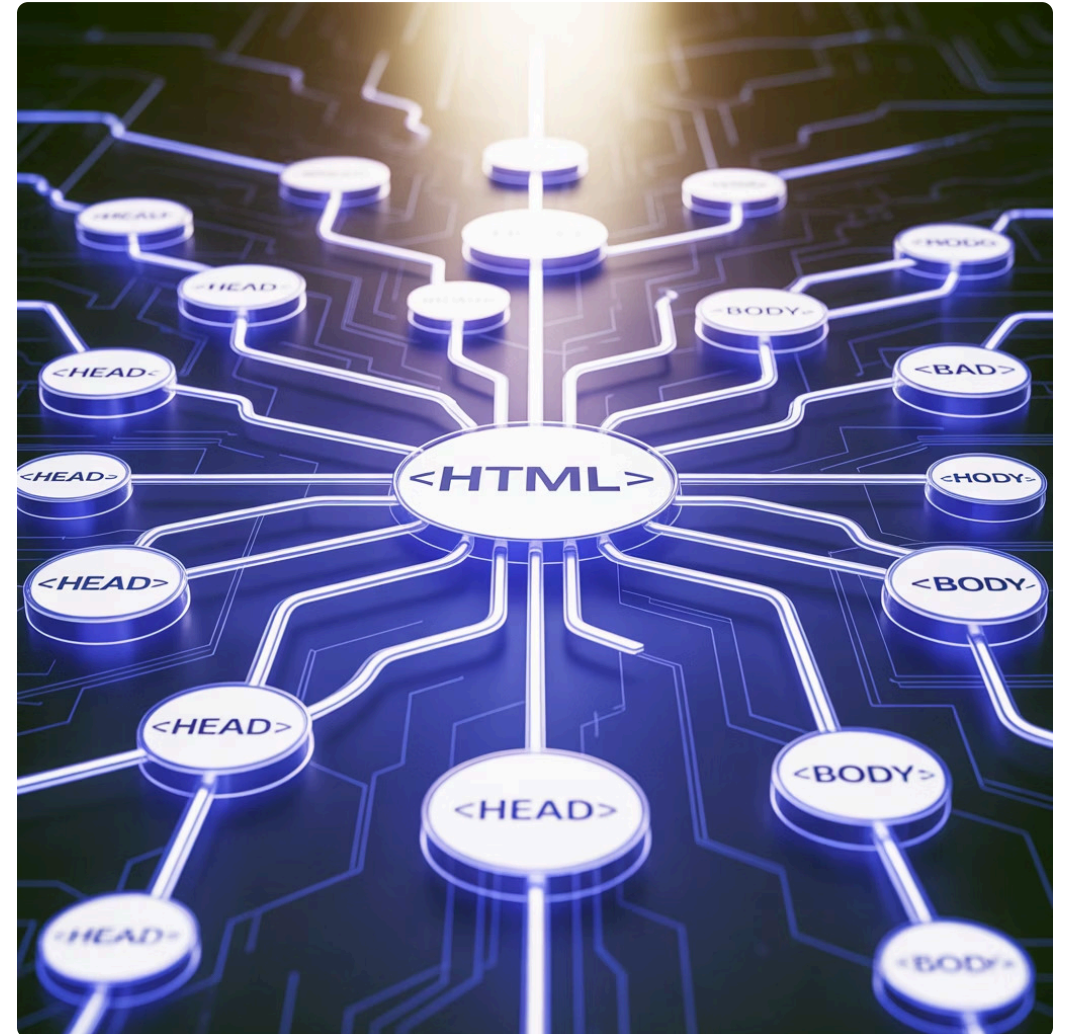
Manipulación del DOM en JavaScript

Aprende a transformar páginas web dinámicamente usando JavaScript para manipular el Document Object Model (DOM) sin necesidad de eventos. Domina las técnicas fundamentales para crear interfaces interactivas.

¿Qué es el DOM?

El **DOM (Document Object Model)** es la representación estructurada de un documento HTML como un árbol de nodos interconectados. Cada elemento HTML, atributo y texto se convierte en un nodo que JavaScript puede manipular.

Esta estructura jerárquica permite acceder a cualquier parte de la página web y modificarla dinámicamente, desde cambiar textos hasta crear nuevos elementos completos.



Selección de Elementos

Antes de manipular cualquier elemento, debemos seleccionarlo correctamente. JavaScript nos proporciona varios métodos potentes para localizar elementos en el DOM:

getElementById()

Selecciona un elemento único por su atributo `id`. Es el método más rápido y eficiente.

```
const titulo =  
document.getElementById("titulo");
```

querySelector()

Utiliza selectores CSS para encontrar el primer elemento que coincida con el patrón especificado.

```
const primerParrafo =  
document.querySelector("p");
```

querySelectorAll()

Devuelve todos los elementos que coinciden con el selector CSS en forma de `NodeList`.

```
const items =  
document.querySelectorAll("li");
```



Modificación de Contenido

Una vez seleccionados los elementos, podemos cambiar su contenido de diferentes maneras según nuestras necesidades:

1

textContent

Modifica únicamente el texto del elemento, interpretando todo como texto plano sin procesar etiquetas HTML.

```
titulo.textContent = "Nuevo título de la página";
```

2

innerHTML

Permite cambiar tanto texto como estructura HTML, procesando y renderizando las etiquetas HTML incluidas.

```
primerParrafo.innerHTML = "<b>Este texto está en negrita</b>";
```

Gestión de Clases CSS

La propiedad `classList` nos proporciona métodos elegantes para manipular las clases CSS de los elementos, permitiendo cambios visuales dinámicos:

+

`classList.add("clase")`

Agrega una nueva clase CSS al elemento sin eliminar las clases existentes.

—

`classList.remove("clase")`

Elimina una clase específica del elemento, manteniendo las demás intactas.

⌂

`classList.toggle("clase")`

Alterna la presencia de una clase: la agrega si no existe o la elimina si ya está presente.

```
titulo.classList.add("resaltado");  
primerParrafo.classList.toggle("oculto");
```




Eliminación de Elementos

Eliminar elementos del DOM es tan importante como crearlos. El método `remove()` nos permite eliminar cualquier elemento de forma limpia y eficiente:

Eliminación directa

Llamamos al método `remove()` directamente sobre el elemento que queremos eliminar.

```
elemento.remove();
```

Eliminación del último hijo

Podemos eliminar el último elemento hijo usando `lastElementChild.remove()`.

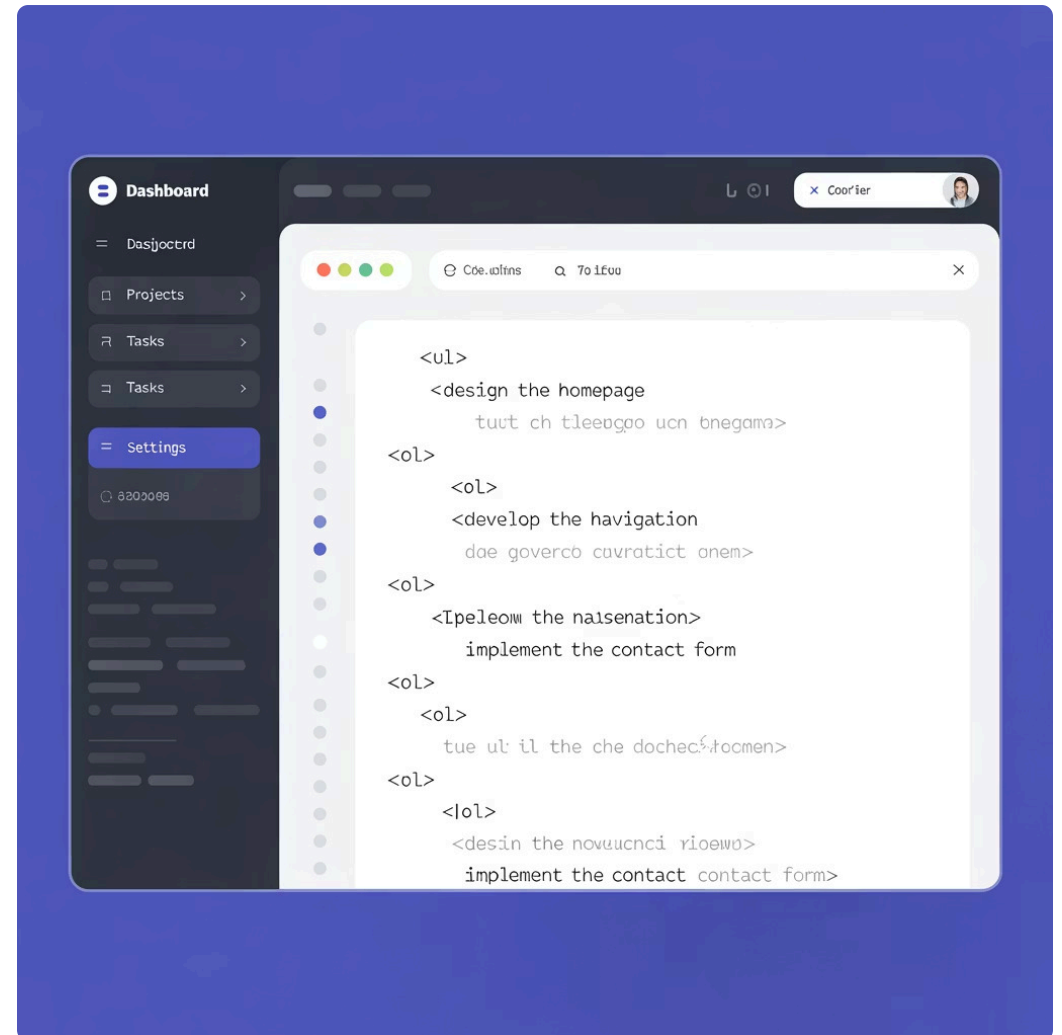
```
lista.lastElementChild.remove();
```

Ejemplo Práctico: Lista de Tareas

Veamos cómo aplicar todos estos conceptos creando una lista de tareas dinámica. Partiremos de una estructura HTML básica y la expandiremos usando JavaScript:

HTML inicial:

```
<h1>Lista de Tareas</h1>
<ul id="lista">
  <li>Tarea 1</li>
  <li>Tarea 2</li>
</ul>
```



Implementación del Código

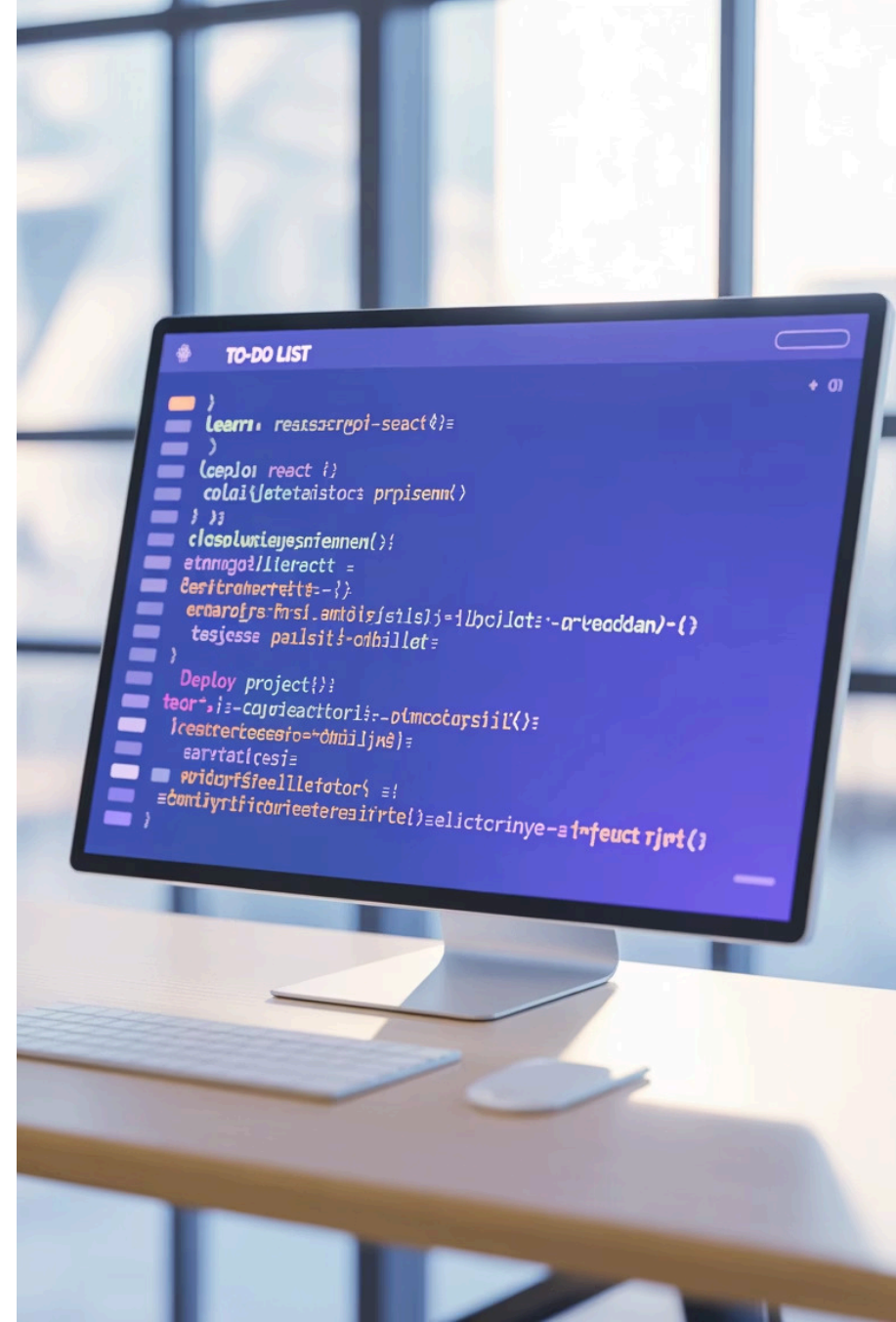
Ahora implementemos la lógica JavaScript para manipular nuestra lista de tareas, agregando nuevos elementos y eliminando otros:

```
const lista = document.getElementById("lista");

// Agregar tres nuevas tareas
for (let i = 3; i <= 5; i++) {
  const li = document.createElement("li");
  li.textContent = `Tarea ${i}`;
  lista.appendChild(li);
}

// Eliminar la última tarea
lista.lastElementChild.remove();
```

Este código demuestra cómo crear elementos dinámicamente usando un bucle y cómo eliminar elementos específicos del DOM.



Resultado Final

1

Estado Inicial

Lista con 2 tareas: "Tarea 1" y "Tarea 2"

2

Después del Bucle

Lista expandida con 5 tareas: desde "Tarea 1" hasta "Tarea 5"

3

Después de Eliminar

Lista final con 4 tareas: "Tarea 1", "Tarea 2", "Tarea 3", "Tarea 4"

¡Felicidades! Has aprendido los fundamentos de la manipulación del DOM sin eventos. Estos conocimientos te permitirán crear interfaces web dinámicas y interactivas de manera eficiente y profesional.

